

Natural Computing SoSe25

Example Exam A

Please mind the following:

- Fill in your **personal information** in the fields below.
- You may use an unmarked dictionary during the exam. **No** additional tools (like calculators, e.g.) might be used.
- Fill in all your answers **directly into this exam sheet**. You may use the backside of the individual sheets or ask for additional paper. In any case, make sure to mark **clearly** which question you are answering. Do not use pens with green or red color or pencils for writing your answers. You may give your answers in both **German and English**.
- Points annotated for the individual tasks only serve as a preliminary guideline.
- At the end of the exam hand in your **whole exam sheet**. Please make sure you do not undo the binder clip!
- Please place your LMU-Card or student ID as well as photo identification (“Lichtbildausweis”) clearly visible on the table next to you. Note that we need to check your data with the data you enter on this cover sheet.

Time Limit: 90 minutes

First Name:																											
Last Name:																											
Matriculation Number:																											
<table border="1"><tr><td>Topic 1</td><td>max. 16 pts.</td><td>pts.</td></tr><tr><td>Topic 2</td><td>max. 11 pts.</td><td>pts.</td></tr><tr><td>Topic 3</td><td>max. 15 pts.</td><td>pts.</td></tr><tr><td>Topic 4</td><td>max. 14 pts.</td><td>pts.</td></tr><tr><td>Topic 5</td><td>max. 14 pts.</td><td>pts.</td></tr><tr><td>Topic 6</td><td>max. 12 pts.</td><td>pts.</td></tr><tr><td>Topic 7</td><td>max. 8 pts.</td><td>pts.</td></tr><tr><td colspan="2">Total max. 90 pts.</td><td>pts.</td></tr></table>				Topic 1	max. 16 pts.	pts.	Topic 2	max. 11 pts.	pts.	Topic 3	max. 15 pts.	pts.	Topic 4	max. 14 pts.	pts.	Topic 5	max. 14 pts.	pts.	Topic 6	max. 12 pts.	pts.	Topic 7	max. 8 pts.	pts.	Total max. 90 pts.		pts.
Topic 1	max. 16 pts.	pts.																									
Topic 2	max. 11 pts.	pts.																									
Topic 3	max. 15 pts.	pts.																									
Topic 4	max. 14 pts.	pts.																									
Topic 5	max. 14 pts.	pts.																									
Topic 6	max. 12 pts.	pts.																									
Topic 7	max. 8 pts.	pts.																									
Total max. 90 pts.		pts.																									

1 Game of Life

16pts

The following definition was given in the lecture.

Definition 1 (Conway's game of life (standard)). Let $G = (V, E)$ be a grid graph with vertices V and (undirected) edges $E \subseteq V \times V$ with a fixed degree of 8 for all nodes. We define $surroundings : V \rightarrow \wp(V)$ via

$$surroundings(v) = \{w \mid (v, w) \in E\}$$

so that $|surroundings(v)| = 8$ and $v \notin surroundings(v)$ for all $v \in V$. A state $x \in \mathcal{X}$ is a mapping of vertices to the labels $\{dead, alive\}$, i.e., the state space $\mathcal{X}$ is given via $\mathcal{X} = (V \rightarrow \{dead, alive\})$. Let x_t be a state that exists at time step $t \in \mathbb{N}$. We define

$$|v|_{x_t} = |\{w \mid w \in surroundings(v) \wedge x_t(w) = alive\}|.$$

In the game of life, the evolution of a state x_t to its subsequent state x_{t+1} is given deterministically via

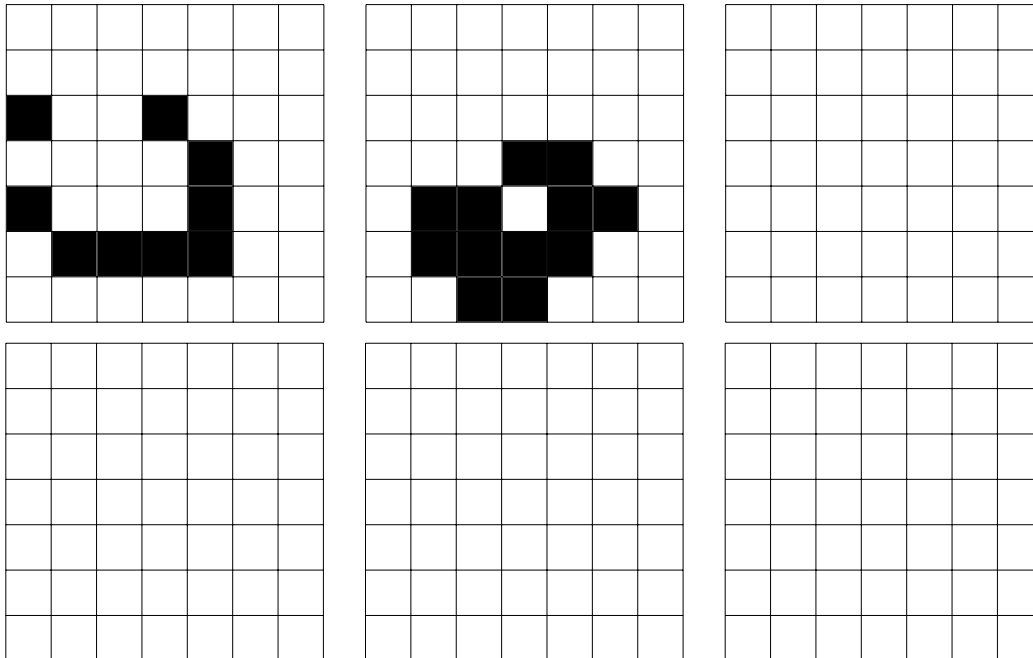
$$x_{t+1}(v) = \begin{cases} dead & \text{if } |v|_{x_t} \leq 1, \\ x_t(v) & \text{if } |v|_{x_t} = 2, \\ alive & \text{if } |v|_{x_t} = 3, \\ dead & \text{if } |v|_{x_t} \geq 4, \end{cases}$$

for all $v \in V$. A tuple (G, x_0) is called an instance of the game of life for initial state $x_0 \in \mathcal{X}$.

(a) Briefly describe the concept of a *spaceship* pattern in the game of life. Give one property that a *spaceship* shares with patterns from the class of *oscillators* and one property that differentiates those two? (3pts)

(b) Continue evolving the pattern given below in the game of life for three more steps. The pattern on the final step definitely proves that we are looking at a *spaceship*. Briefly explain why *this* pattern is a *spaceship*. You may use the empty grids following the pattern to draw its subsequent states. (10pts)

Hint: Using symmetry and the *spaceship* property, you might be able to draw some states quicker than going cell by cell.



Note: There is free space in the Appendix at the back of the exam, should you need more than the one extra grid space printed here. Mark clearly on the task's page, if the solution to a task is to be found there, and mark clearly in the Appendix, which task a solution belongs to.

In the game of life, the *speed* property denotes the number of cells that the pattern (e.g., a *spaceship*) moves on average per generation. Since no object on the board could possibly move at a speed of greater than 1 cell per step (with Moore neighborhood, cf. Def. 1), this speed is simply denoted by c , while other speeds are represented as fractions of c .

Example: The *glider* pattern moves diagonally by 1 cell every 4 steps, thus on average it moves $1/4$ cell per generation, i.e., with a speed of $c/4$.

(c) State the speed of the *spaceship* pattern in task (b). Briefly explain your reasoning. (3pts)

2 Cellular Automata

11pts

The following definitions (Defs. 2 and 3) were given in the lecture.

Definition 2 (cellular automaton). Let $G = (V, E)$ be a graph with vertices V and edges $E \subseteq V \times V$. Let $neighborhood : V \rightarrow V^{d+1}$ for some neighborhood degree $d \in \mathbb{N}$ be a function that returns an ordered vector of neighbors of a given node v , always including v itself. A state $x \in \mathcal{X}$ is a mapping of vertices to the values $\{0, 1\}$, i.e., the state space $\mathcal{X}$ is given via $\mathcal{X} = (V \rightarrow \{0, 1\})$. Let x_t be a state that exists at time step $t \in \mathbb{N}$. The evolution of a state x_t to its subsequent state x_{t+1} is given deterministically via a function $f : \{0, 1\}^{d+1} \rightarrow \{0, 1\}$ so that

$$x_{t+1}(v) = f(x_t(u_1), \dots, x_t(u_{d+1}))$$

where $\vec{u} = \langle u_1, \dots, u_{d+1} \rangle = neighborhood(v)$.

A tuple (G, f, x_0) is called a cellular automaton with initial state $x_0 \in \mathcal{X}$.

Definition 3 (1D cellular automaton). A cellular automaton (G, f, x_0) is called a 1D cellular automaton iff all vertices have exactly one incoming edge (“left neighbor”) and one outgoing edge (“right neighbor”) and $neighborhood(v) = \langle u, v, w \rangle$ where $(u, v) \in E$ and $(v, w) \in E$ and thus $d = 2$. Subsequently, we usually write

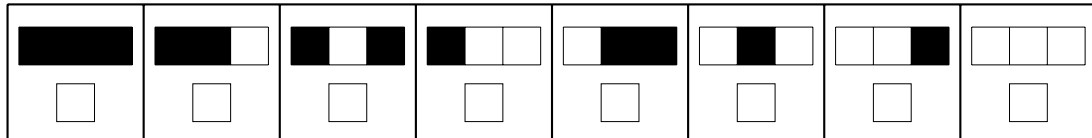
$$x_{t+1}(v) = f(x_t(u), x_t(v), x_t(w))$$

for the evolution where u is the left neighbor of v and w is the right neighbor of v .

Note: There is free space in the Appendix at the back of the exam should you need more grid space. Mark clearly on the task’s page, if the solution to a task is to be found there, and mark clearly in the Appendix, which task a solution belongs to.

A 1D cellular automaton is often given via a *rule* $n \in [0; 255] \subset \mathbb{N}$ where each digit in n ’s binary representation encodes one output of f so that the digit for the base value 128 encodes the output of $f(1, 1, 1)$, 64 for $f(1, 1, 0)$, and so on, sorted by decreasing numerical value.

(a) Fill in the rulestring 150, then evolve this initial configuration by hand for three generations. (4pts)



(b) For the following initial configuration and the rulestring 150, draw one configuration below that does not result from the initial configuration through the application of rulestring 150. Briefly state your reasoning. (2pts)

We now consider the following new definition:

Definition 4 (fancy exam 1D cellular automaton). A cellular automaton (G, f', x'_0) is called a *fancy exam* 1D cellular automaton iff all vertices have exactly one incoming edge (“left neighbor”) and one outgoing edge (“right neighbor”) and $neighborhood(v) = \langle r, u, v, w, z \rangle$ where r, u, v, w, z are all mutually different and $(r, u), (u, v), (v, w), (w, z) \in E$ with and thus $d = 4$.

(c) Write down the adjusted formula to compute x'_{t+1} for the *fancy exam 1D cellular automaton* given the current state x'_t , which uses f' instead of f in Defs. 2 and 3. Give a brief mathematical definition for f' such that calling f' results in a state where $x'_{t+1}(v) = 1$ if and only if v has an even amount of alive neighbors. (5pts)

3 Quantum Computing

15pts

The following definitions were given in the lecture.

Definition 5 (quantum state). Let $\mathcal{X}$ be an arbitrary state space. Let $\mathcal{Q}(\mathcal{X})$ be the space of superposition states with bases in $\mathcal{X}$, i.e., each $q \in \mathcal{Q}(\mathcal{X})$ has the form $q = \sum_{x \in \mathcal{X}} c_x |x\rangle$ where $c_x \in \mathbb{C}$ for any $x \in \mathcal{X}$ is called an amplitude so that $\sum_{x \in \mathcal{X}} |c_x|^2 = 1$ and $|x\rangle$ (read “ket x ”) denotes a vector corresponding to the state $x \in \mathcal{X}$.

We use the following terminology:

- If for more than one $x \in \mathcal{X}$ it holds that $c_x \neq 0$, then q is called a *superposition* state.
- If $\mathcal{X} = \mathcal{X}_A \times \mathcal{X}_B$, i.e., $\mathcal{X}$ can be constructed as a product of two other state spaces $\mathcal{X}_A, \mathcal{X}_B$, then it follows that q has the form

$$q = \sum_{\substack{x_A \in \mathcal{X}_A, \\ x_B \in \mathcal{X}_B}} c_{x_A, x_B} |x_A\rangle \otimes |x_B\rangle$$

where $\otimes$ is the tensor product. If q cannot be written as

$$q = \left(\sum_{x_A \in \mathcal{X}_A} c_{x_A} |x_A\rangle \right) \otimes \left(\sum_{x_B \in \mathcal{X}_B} c_{x_B} |x_B\rangle \right),$$

then we call q an *entangled* state.

- If $\mathcal{X} = \{0, 1\}$ (i.e., any $x \in \mathcal{X}$ is a bit), then any $q \in \mathcal{Q}(\mathcal{X})$ is called a *qubit* and can be written as

$$q = c_0 |0\rangle + c_1 |1\rangle$$

for $c_0, c_1 \in \mathbb{C}$ with $|c_0|^2 + |c_1|^2 = 1$.

- If $\mathcal{X} = \{0, 1\}^n$ for any $n \in \mathbb{N}$ (i.e., any $x \in \mathcal{X}$ is a bit register), then any $q \in \mathcal{Q}(\mathcal{X})$ is called a *qubit register* of size n and can be written as

$$q = \sum_{i=0}^{2^n-1} c_i |i\rangle = c_{0\dots 0} |0\dots 0\rangle + \dots + c_{1\dots 1} |1\dots 1\rangle$$

where we usually denote i in binary as illustrated above.

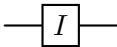
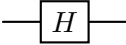
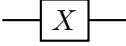
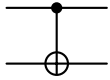
- When we *measure* a quantum state $q \in \mathcal{Q}(\mathcal{X})$, we generate classical state $M(q) \in \mathcal{X}$ so that for all $x \in \mathcal{X}$ it holds that

$$\Pr(M(q) = x) = |c_x|^2.$$

(a) **Show why $|a\rangle = 0.6 \cdot |101\rangle + 0.6 \cdot |010\rangle$ is not a valid quantum state.** (3pts)

(b) **State if $|b\rangle = \frac{1}{\sqrt{3}} \cdot |00\rangle + \frac{1}{\sqrt{3}} \cdot |01\rangle + \frac{1}{\sqrt{3}} \cdot |10\rangle$ is in an entangled quantum state or not. Show your answer.** (5pts)

(c) **Give the probability of observing $|01\rangle$ when measuring the state $|b\rangle$, i.e., compute $\Pr(M(|b\rangle) = |01\rangle) = \langle 01 | b \rangle = \dots?$** (1pts)

Operator	Gate	Matrix
Identity (I)		$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$
Hadamard (H)		$\frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$
Pauli-X (X)		$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$
Controlled-X (CNOT)		$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$

(d) Using two of the quantum operators we have covered in the lecture (see above), construct a quantum circuit that prepares the following state $|abc\rangle$ by drawing the operator gates on the qubit wire template below. Briefly explain your answer. (6pts)

Note: We draw the leftmost qubit in the state notation as the topmost wire.

Prepare the state:

$$|abc\rangle = \frac{1}{\sqrt{2}} |000\rangle + \frac{1}{\sqrt{2}} |110\rangle$$

$|a\rangle := |0\rangle$ _____

$|b\rangle := |0\rangle$ _____

$|c\rangle := |0\rangle$ _____

4 (Population-Based) Optimization

14pts

We give a shortened version of the definition for optimization processes from the lecture and then introduce a new optimization algorithm called *simplified firefly optimization*.

Definition 6 (optimization (shortened)). Let $\mathcal{X}$ be an arbitrary state space. Let $\mathcal{T}$ be an arbitrary set called target space and let $\leq$ be a total order on $\mathcal{T}$. A total function $\tau : \mathcal{X} \rightarrow \mathcal{T}$ is called target function. Optimization (minimization/maximization) is the procedure of searching for an $x \in \mathcal{X}$ so that $\tau(x)$ is optimal (minimal/maximal). Unless stated otherwise, we assume minimization.

An optimization run of length $g+1$ is a sequence of states $\langle x_t \rangle_{0 \leq t \leq g}$ with $x_t \in \mathcal{X}$ for all t .

Let $e : \langle \mathcal{X} \rangle \times (\mathcal{X} \rightarrow \mathcal{T}) \rightarrow \mathcal{X}$ be a possibly randomized or non-deterministic function so that the optimization run $\langle x_t \rangle_{0 \leq t \leq g}$ is produced by calling e repeatedly, i.e., $x_{t+1} = e(\langle x_u \rangle_{0 \leq u \leq t}, \tau)$ for all t , $1 \leq t \leq g$, where x_0 is given externally (e.g., $x_0 =_{\text{def}} 42$) or chosen randomly (e.g., $x_0 \sim \mathcal{X}$). An optimization process is a tuple $(\mathcal{X}, \mathcal{T}, \tau, e, \langle x_t \rangle_{0 \leq t \leq g})$.

Definition 7 (population-based optimization). Let $\mathcal{X}$ be a state space. Let $\mathcal{T}$ be a target space with total order $\leq$. Let $\tau : \mathcal{X} \rightarrow \mathcal{T}$ be a target function. A tuple $\mathcal{E} = (\mathcal{X}, \mathcal{T}, \tau, E, \langle X_t \rangle_{0 \leq t \leq g})$ is a population-based optimization process iff $X_t \in \wp^*(\mathcal{X})$ for all t and $E : \langle \wp^*(\mathcal{X}) \rangle \times (\mathcal{X} \rightarrow \mathcal{T}) \rightarrow \wp^*(\mathcal{X})$ is a possibly randomized, non-deterministic, or further parametrized function so that the population-based optimization run is produced by calling E repeatedly, i.e., $X_{t+1} = E(\langle X_u \rangle_{0 \leq u \leq t}, \tau)$ where X_0 is given externally or chosen randomly.

Algorithm 1 (simplified firefly optimization^a). Let $\mathcal{E} = (\mathcal{X}, \mathcal{T}, \tau, E, \langle X_t \rangle_{0 \leq t \leq g})$ be a population-based optimization process. We assume $\mathcal{X} = [-10; 10] \subset \mathbb{R}$ and $\mathcal{T} = [0; 100] \subset \mathbb{R}$ for minimization. For $x \notin [-10; 10]$ we assume that we can still call τ so that $\tau(x) = \infty$. The process $\mathcal{E}$ continues via simplified firefly optimization iff E is given via

$$E(\langle X_u \rangle_{0 \leq u \leq t}, \tau) = X_{t+1} = \begin{aligned} &\{x_{i,t} + \beta \cdot (x_t^* - x_{i,t}) && : x_{i,t} \in X_t \wedge \tau(x_{i,t}) > \tau(x_t^*)\} \\ &\cup \{x_{i,t} && : x_{i,t} \in X_t \wedge \tau(x_{i,t}) \leq \tau(x_t^*)\} \end{aligned}$$

where $X_t = \{x_{i,t}\}_{i=1}^n$ are the (initial) positions of n fireflies in $\mathcal{X}$ at step t , β is a fixed step size parameter ($\beta \in (0; 1] \subset \mathbb{R}$), and x_t^* is the best individual at time t , i.e.,

$$x_t^* = \arg \min_{x_t \in X_t} \{\tau(x_t)\}.$$

^aAdapted from Yang, X.-S. (2008)

(a) Algorithm 1 is based on an observation that fireflies are attracted to the light of other fireflies. We assume that, if the light of a firefly is more intense, then other fireflies will be drawn more strongly towards it. **Briefly explain how this behavior is represented in Algorithm 1. Which value used in the algorithm corresponds (maybe inversely) to *light intensity*?** (4pts).

(b) Let the target function to be minimized be $\tau(x) = (x - 2)^2$. Also let $\beta = 0.5$ and let the initial state of the positions $X_0 = \{0, 4, 6\}$. **Execute *simplified firefly optimization* (cf. Algorithm 1) in this setting for two time steps, i.e., compute X_1, X_2 . State the global optimum of τ and whether it is reached after two optimization steps.** (10pts)

Hint: If two elements are equal for $\arg \min$, our version of $\arg \min$ simply picks one of them at random, which means that, when you execute the algorithm, you can choose one of them at will.

5 Artificial Chemistries / Soups

14pts

Recall the following definition from the lecture:

Definition 8 (artificial chemistry, soup). Let $\mathcal{X}$ be a state space. Let $\mathcal{R}$ be a set of reaction rules $R \in \mathcal{R}$ where each R is a function $R : \mathcal{X}_R^{\text{dom}} \rightarrow \mathcal{X}_R^{\text{cod}}$ for $\mathcal{X}_R^{\text{dom}}, \mathcal{X}_R^{\text{cod}} \subseteq \wp^*(\mathcal{X})$. Let $\mathcal{A}(X, \mathcal{R})$ be the set of all *reaction materials* from $X \in \wp^*(\mathcal{X})$ for all rules in $\mathcal{R}$, i.e.,

$$\mathcal{A}(X, \mathcal{R}) = \{(X', R) \mid X' \subseteq X \wedge X' \in \mathcal{X}_R^{\text{dom}} \wedge R \in \mathcal{R}\}.$$

Let $A : \wp^*(\mathcal{X}) \rightarrow \wp^*(\mathcal{X}) \times \mathcal{R}$ be a possibly randomized or non-deterministic function that takes a multi-set $X \in \wp^*(\mathcal{X})$ and returns a multi-set of reactants and a reaction rule to perform so that $A(X) \in \mathcal{A}(X, \mathcal{R})$. The tuple $(\mathcal{X}, \mathcal{R}, A)$ defines an artificial chemistry. A multi-set $X \in \wp^*(\mathcal{X})$ of an artificial chemistry is also called soup; each element $x \in X$ is also called a particle. The evolution of a soup for g generations is a sequence $\langle X_0, \dots, X_g \rangle$ so that X_0 is given as the initial soup and, for all t with $0 \leq t \leq g-1$

$$X_{t+1} = (X_t \setminus X') \uplus R(X')$$

where $(X', R) = A(X_t)$.

Reaction rules are usually written in the form $R = \sum_i x_i \rightarrow \sum_j y_j$ for $x_i, y_j \in \mathcal{X}$, $i, j \in \mathbb{N}$, $\{x_i\}_i \in \mathcal{X}_R^{\text{dom}}$, $\{y_j\}_j \in \mathcal{X}_R^{\text{cod}}$, and $R(\{x_i\}_i) = \{y_j\}_j$.

Note: We use the following (very intuitive) shortcut in notation: The expression $n \cdot x$ for some $n \in \mathbb{N}$ and $x \in \mathcal{X}$ is defined to be $\underbrace{x + \dots + x}_{n \text{ times}}$.

We now define the artificial chemistry $\mathcal{C} = (\mathcal{X}, \mathcal{R}, A)$ by defining:

Let $\mathcal{X} = \{\text{EMU}, \text{EGG}, \text{DINGO}, \text{HUNGRYCROCODILE}, \text{FULLCROCODILE}\}$.

Let $\mathcal{R} = \{R_1, R_2, R_3, R_4, R_5, R_{6,x}, R_7\}$ with

$$\begin{array}{ll} R_1 = 2 \cdot \text{EMU} & \rightarrow 2 \cdot \text{EMU} + 5 \cdot \text{EGG} \\ R_2 = \text{EGG} & \rightarrow \text{EMU} \\ R_3 = \text{DINGO} + \text{EMU} & \rightarrow \text{DINGO} \\ R_4 = \text{DINGO} + \text{EGG} & \rightarrow \text{DINGO} \\ R_5 = 2 \cdot \text{DINGO} + 3 \cdot \text{EMU} & \rightarrow 2 \cdot \text{DINGO} \\ R_{6,x} = \text{HUNGRYCROCODILE} + x & \rightarrow \text{FULLCROCODILE} \\ R_7 = \text{FULLCROCODILE} & \rightarrow \text{HUNGRYCROCODILE} \end{array}$$

for all $x \in \mathcal{X} \setminus \{\text{EGG}\}$.

Let $A(X) \sim \mathcal{A}(X, \mathcal{R})$.

(a) Give a possible evolution of the soup

$$X_0 = \{3 \cdot \text{EMU}, 2 \cdot \text{DINGO}, \text{HUNGRYCROCODILE}\}$$

for as many steps as necessary until anything but one HUNGRYCROCODILE disappears, i.e., until the current soup at some time t is $X_t = \{\text{HUNGRYCROCODILE}\}$.

(7pts)

(Hint: Instead of random choices, you can decide which rules to apply to make the process quicker. Use the table below.)

time step	chosen rule	current soup
0	none	$\{3 \cdot \text{EMU}, 2 \cdot \text{DINGO}, \text{HUNGRYCROCODILE}\}$
1		
2		
3		
4		
5		
6		
7		

(b) Based on the artificial chemistry $\mathcal{C} = (\mathcal{X}, \mathcal{R}, A)$ we now define a new artificial chemistry $\mathcal{C}' = (\mathcal{X}', \mathcal{R}', A')$ by defining:

Let $\mathcal{X}' = \mathcal{X} \cup \{\text{KANGAROO}(x) : x \in \mathcal{X}' \cup \{0\}\}$.

Let $\mathcal{R}' = \mathcal{R} \cup \{R_{8,-}^{\leftarrow}, R_{8,-}^{\rightarrow}, R_{9,-}\}$ where

$$R_{8,x} = \text{KANGAROO}(0) + x \quad \Rightarrow \text{KANGAROO}(x)$$

$$R_{9,x} = 2 \cdot \text{DINGO} + \text{KANGAROO}(x) \quad \rightarrow 2 \cdot \text{DINGO} + x$$

for all $x \in \mathcal{X}'$. Let $A'(X) \sim \mathcal{A}(X, \mathcal{R}')$.

Give a possible evolution of the soup

$$X'_0 = \{\text{KANGAROO}(\text{KANGAROO}(0)), 2 \cdot \text{DINGO}, \text{HUNGRY CROCODILE}\}$$

for as many steps as necessary until no particle from the original state space $\mathcal{X}$ remains, i.e., until for the current soup X'_t at some time t it holds that $\forall x \in X'_t : x \notin \mathcal{X}$ (while naturally $\forall x \in X'_t : x \in \mathcal{X}'$ always remains true). Correctly annotate the direction in which $R_{8,-}$ is used! (7pts)

(Hint: Instead of random choices, you can decide which rules to apply to make the process quicker. Use the table below.)

time step	chosen rule	current soup
0	none	$\{\text{KANGAROO}(\text{KANGAROO}(0)), 2 \cdot \text{DINGO}, \text{HUNGRY CROCODILE}\}$
1		
2		
3		
4		
5		
6		

6 Evolutionary Computing

12pts

If necessary, you can recollect the definition for evolutionary algorithms as used in the lecture on this page. (Hint: Try the tasks first before reading this lengthy definition again.)

Algorithm 2 (typical evolutionary algorithm). Let $\mathcal{E} = (\mathcal{X}, \mathcal{T}, \phi, E, \langle X_u \rangle_{0 \leq u \leq t})$ be a population-based optimization process.

- Let $\sigma_N^{survivors}, \sigma_N^{parents} : \wp^*(\mathcal{X}) \rightarrow \wp^*(\mathcal{X})$ be (possibly randomized or non-deterministic) selection functions that return $N \in \mathbb{N}$ individuals, i.e., $|\sigma_N(X)| = N$ and $\sigma_N(X) \subseteq X$ for all X . They may possibly depend on $\mathcal{E}$ (most commonly ϕ). Let $\varrho_N^{mutants} : \wp^*(\mathcal{X}) \rightarrow \wp^*(\mathcal{X})$ be a special selection function that, given a population X , returns a random subset $X' \subseteq X$ so that $|X'| = N$. Note that $\varrho_{|X|}(X) = X$ for all X .
- Let $mutate : \mathcal{X} \rightarrow \mathcal{X}$ be a (possibly randomized or non-deterministic) single-individual mutation function. We write $mutate[_] : \wp^*(\mathcal{X}) \rightarrow \wp^*(\mathcal{X})$ for the per-individual application of $mutate$, i.e., $mutate[X] = \{mutate(x) : x \in X\}$.
- Let $recombine : \mathcal{X} \times \mathcal{X} \rightarrow \mathcal{X}$ be a (possibly randomized or non-deterministic) two-parent recombination function. We write $recombine[_] : \wp^*(\mathcal{X}) \rightarrow \wp^*(\mathcal{X})$ for the random-pairing application of $recombine$, i.e., $recombine[X] = \{recombine(x_1, x_2)\} \uplus recombine[X \setminus \{x_1, x_2\}]$ with $x_1, x_2 \sim X$ and $recombine[\{x\}] = \emptyset$ for all x as well as $recombine[\emptyset] = \emptyset$.
- Let $migrate : \emptyset \rightarrow \mathcal{X}$ be a (possibly randomized or non-deterministic) individual creation function. We write $migrate_N : \emptyset \rightarrow \wp^*(\mathcal{X})$ with $N \in \mathbb{N}$ for the iterated application of $migrate$, i.e., $migrate_N[] = \{migrate()\} \uplus migrate_{N-1}[]$ with $migrate_0[] = \emptyset$.

The process $\mathcal{E}$ continues via a typical evolutionary algorithm if E has the form

$$E(\langle X_u \rangle_{0 \leq u \leq t}, \phi) = X_{t+1} = \sigma_{|X_t|}^{survivors}(X_t \uplus mutate[\varrho_M^{mutants}(X_t)] \uplus recombine[\sigma_{2C}^{parents}(X_t)] \uplus migrate_H[])$$

where $M \in \mathbb{N}$ is the number of mutants produced per generation, $C \in \mathbb{N}$ is the number of children produced per generation, and $H \in \mathbb{N}$ is the number of migrants (sometimes called hypermutants) generated per generation. Especially M is often expressed as a rate for random choice within the population, i.e., $M = \lceil m \cdot |X_t| \rceil$ where $m \in \mathbb{R}$ is called mutation rate.

We now consider starting an emu farm. Somehow, our zoologist friends have convinced us that emus prefer to live in enclosures which are parallelograms. From an aunt we borrowed 1000 steel fence pieces, from which we want to build the ideal parallelogram for our emus — using an evolutionary algorithm. Most fortunately, a breakthrough study by our zoologist friends gave us a working implementation of a function μ , which can compute the emus' parallelogram preference. The function μ has the type

$$\mu : \mathbb{N} \times \mathbb{N} \times \mathbb{A} \rightarrow \mathbb{M}$$

where $\mathbb{A} = [0; 90] \subset \mathbb{R}$ describes the bottom left angle of the parallelogram (as real-valued degrees) and $\mathbb{M} = [-100; +100] \subset \mathbb{R}$ describes the emus' mood when inside the enclosure (as a real value). In the call $\mu(l, m, \alpha) = v$ we thus interpret l to be the length of one side the parallelogram (in fence pieces), m to be the length of the other side of the parallelogram (also in fence pieces), α to be the angle between the sides, and v to be the emus' preference to be maximized. Recall that a parallelogram is fully defined by l, m, α .

(a) Despite what the emus' preference might say, we only have at most 1000 fence pieces to build our enclosure. **Give a total target function $\tau : \mathbb{Z} \times \mathbb{Z} \times \mathbb{A} \rightarrow \mathbb{R}$ that maximizes the emus' preference as specified by μ but also drastically punishes solution candidates that do not allow us to build a correct parallelogram with the fence pieces we have. State if τ is to be maximized or minimized.** (6pts)

(b) A beautiful recombination function, which can be called via $recombine(x_1, x_2)$ on two solution candidates $x_1, x_2 \in \mathcal{X}$, has the following properties:

- The generated child contains some information from both its parents.
- Any information contained in the child has the same chance of stemming from any of the two parents.
- If both parents encode valid parallelograms for the fence pieces we have, so does the child.

Give a beautiful recombination function. (6pts)

7 Mix-Up: Optimization for Soups

8pts

Recall the following definition from the lecture.

Algorithm 3 (simulated annealing). Let $\mathcal{D} = (\mathcal{X}, \mathcal{T}, \tau, e, \langle x_u \rangle_{0 \leq u \leq t})$ be an optimization process. Let $neighbors : \mathcal{X} \rightarrow \wp(\mathcal{X})$ be a function that returns a set of neighbors of a given state $x \in \mathcal{X}$. Let $\kappa : \mathbb{N} \rightarrow \mathbb{R}$ be a temperature schedule, i.e., a function that returns a temperature value for each time step. Let $A : \mathcal{T} \times \mathcal{T} \times \mathbb{R} \rightarrow \mathbb{P}$ with $\mathbb{P} = [0; 1] \subset \mathbb{R}$ be a function that returns an acceptance probability given two target values and a temperature. Commonly, we use $A(Q, Q', K) = e^{\frac{-(Q' - Q)}{K}}$ for $\mathcal{T} \subseteq \mathbb{R}$. The process $\mathcal{D}$ continues via simulated annealing if e is of the form

$$e(\langle x_u \rangle_{0 \leq u \leq t}, \tau) = x_{t+1} = \begin{cases} x'_t & \text{if } \tau(x'_t) \leq \tau(x_t) \text{ or } r \leq A(\tau(x_t), \tau(x'_t), \kappa(t)), \\ x_t & \text{otherwise,} \end{cases}$$

where $x'_t \sim neighbors(x_t)$ and $r \sim \mathbb{P}$ are drawn at random for each call to e .

Consider the soup $\mathcal{C}' = (\mathcal{X}', \mathcal{R}', A')$ from task 5b on this exam. For an Australia exhibit in our zoo, we want to choose an optimal initial soup $X_0^\dagger$ so that all aspects of the Australian wildlife (as defined by $\mathcal{C}'$) can be shown, e.g., a thriving (but not uncontrolled) population of emus. Our preferences for the exhibit are encoded in a given target function $\tau^\dagger : \mathcal{X}^\dagger \rightarrow \mathbb{R}$, which is to be minimized.

(a) In order to find minimal solutions for $\tau^\dagger$, we want to use an optimization process $\mathcal{A} = (\mathcal{X}^\dagger, \mathbb{R}, \tau^\dagger, e^\dagger, \langle x_u^\dagger \rangle_{0 \leq u \leq t})$ which continues via simulated annealing. **Give the state space $\mathcal{X}^\dagger$ for simulated annealing and a suitable function $neighbors$ so that $\mathcal{A}$ can potentially reach any possible initial soup from any initial soup solution candidate with the right number of iterations and choice of parameters.** (6pts)

(b) **Briefly state why giving an easily executable $neighbors$ function for soups in the artificial chemistry $\mathcal{C}'$ from task 5b is much more complex than it would have been for soups in the artificial chemistry $\mathcal{C}$ from task 5a.** (2pts)

Appendix.

On this page, you find additional grid fields if you need them for the tasks in this exam. Mark clearly on the task's page, if the solution to a task is to be found here, and mark clearly on this page here, which task a solution belongs to.

